

DISTRIBUTED AND CLOUD COMPUTING

LAB 3: MPI N-N COMMUNICATIONS & DOCKER

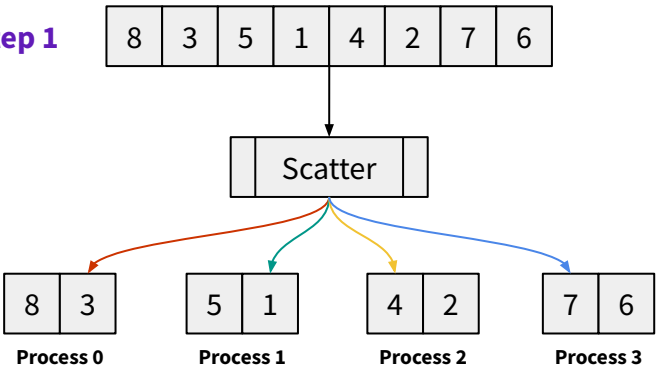
(Module: Message Passing)



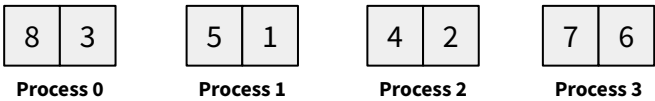
Parallelized Computation - Merge Sort

- Merge sort is a classic divide-and-conquer sorting algorithm
- Process: Divide list into unsorted sub-lists, then sort sub-lists, finally merge all sorted sub-lists.
- Good candidate for parallelize: Sorting of sub-lists are independent!

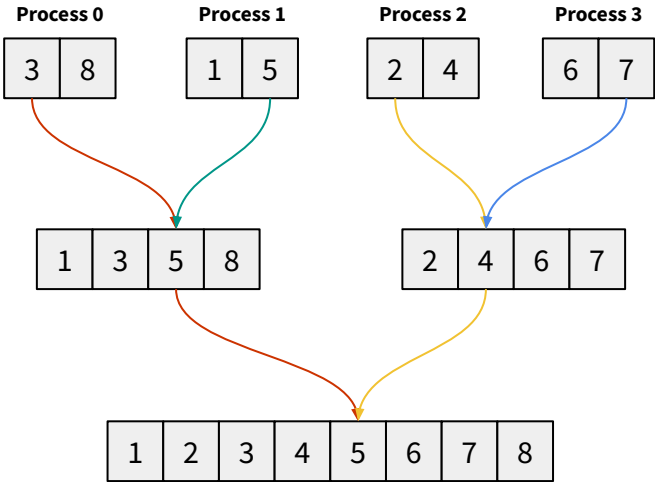
Step 1



Step 2



Step 3

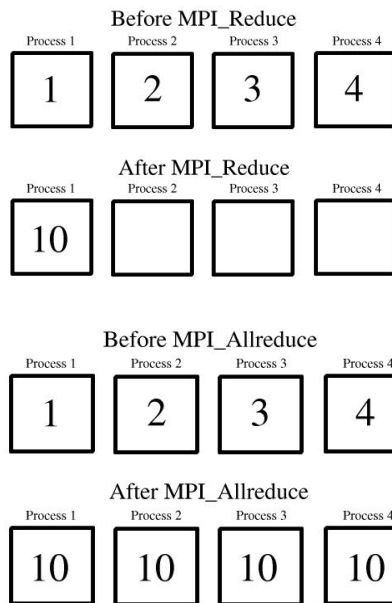


Collective Communication (N-N)

Reduce methods implement a distributed computation using data distributed over all processes

MPI_Reduce: result of computation is gathered by the ROOT

MPI_Allreduce: result of computation is broadcasted to all processes



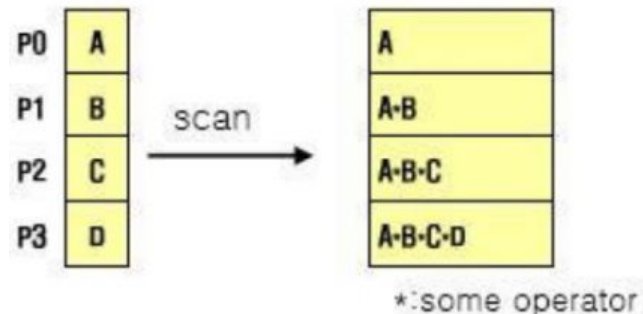
BUILT-IN OPERATIONS IN MPI

- [MPI_MAX] maximum
- [MPI_MIN] minimum
- [MPI_SUM] sum
- [MPI_PROD] product
- [MPI_LAND] logical and
- [MPI_BAND] bit-wise and
- [MPI_LOR] logical or
- [MPI BOR] bit-wise or
- [MPI_LXOR] logical xor
- [MPI_BXOR] bit-wise xor
- [MPI_MAXLOC] max value and location
- [MPI_MINLOC] min value and location

Collective Communication (N-N)

Scan: Each process gets the first k items

- process 1 gets the first item
- process 2 gets the first 2 items

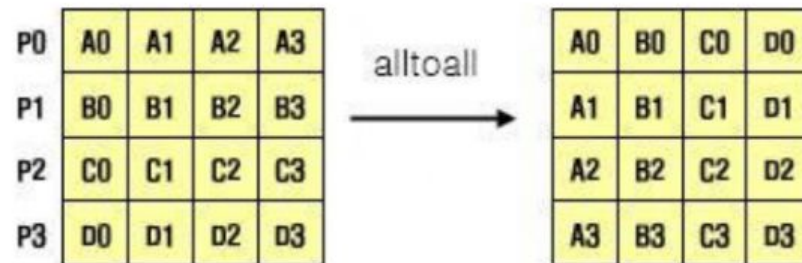


AlltoAll: Each process gets **all** #<rank> items

- process 1 gets all the items with index 1
- process 2 gets all items with index 2

Alternative explanation:

- **It's a transpose operation!**



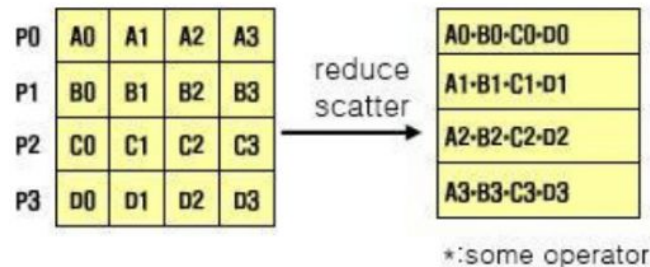
Collective Communication (N-N)

MPI_Reduce_scatter: reduce values from all processes one by one, then scatter the reduced results to all processes.

Another interpretation:

1. Perform an **AllToAll operation!**
2. Locally reduce the values per process.

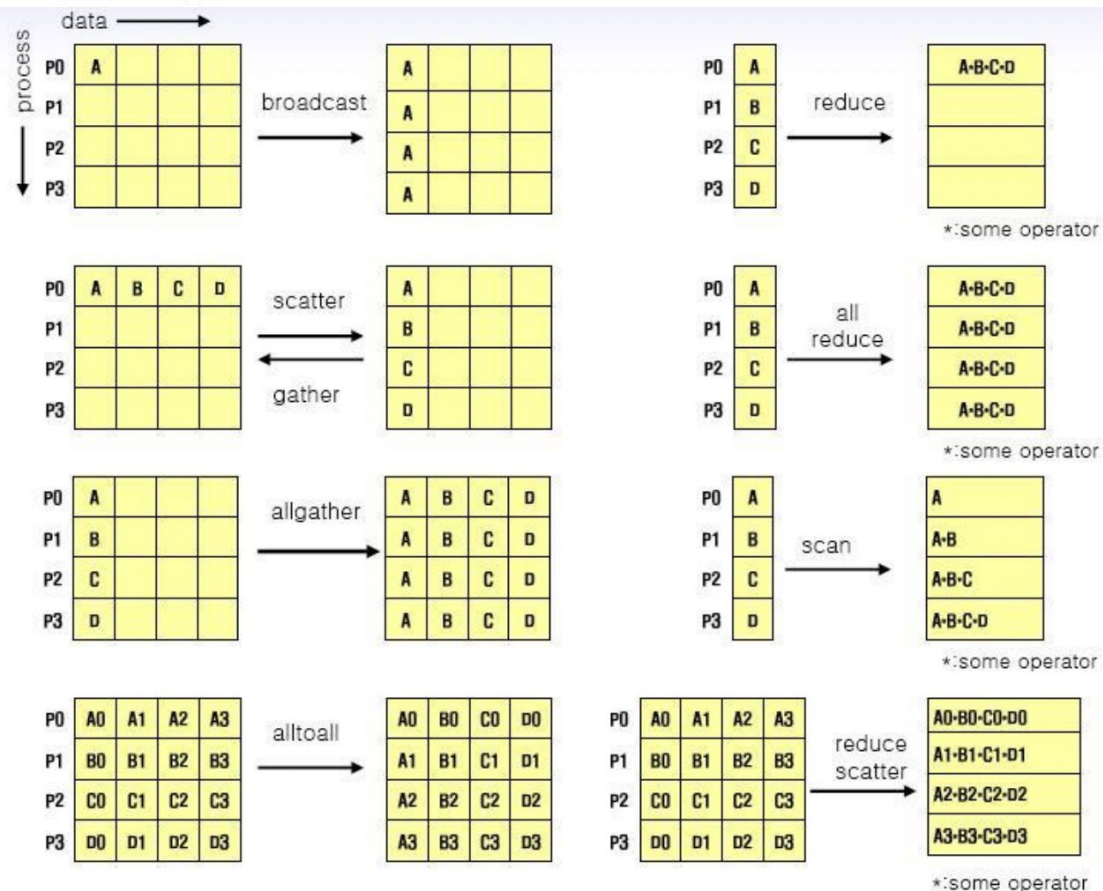
If it helps: you can remember this as “Transpose Reduce”



BUILT-IN OPERATIONS IN MPI

- [MPI_MAX] maximum
- [MPI_MIN] minimum
- [MPI_SUM] sum
- [MPI_PROD] product
- [MPI_LAND] logical and
- [MPI_BAND] bit-wise and
- [MPI_LOR] logical or
- [MPI_BOR] bit-wise or
- [MPI_LXOR] logical xor
- [MPI_BXOR] bit-wise xor
- [MPI_MAXLOC] max value and location
- [MPI_MINLOC] min value and location

Collective communication in one figure



TASK: Distributed Dot Product

PART A: Study the provided MPI examples for N-to-N collective communication

Source files: mpi_NtoN_example.c | mpi_reduce_scatter_example.c | mpi_scan_example.c

PART B: Using 3 workers perform a dot product operation on 2 vectors of length 9

Step 1: Root (0) scatters the vectors over the workers (including self)

Step 2: Workers perform dot product on sub-vectors

$$A = [a_1, a_2, \dots, a_i] \quad B = [b_1, b_2, \dots, b_i] \quad | \quad A \text{ dot } B = \sum_{x=0}^{x=i} a_x * b_x$$

Step 3: Perform a reduction on dot products (MPI_SUM) and print result on root (0)

```
mpicc source.c -o executable_name
```

```
mpirun -np <num_processes> ./executable_name
```


Distributed Dot Product (Observations)

VECTOR SIZE: 800

```

george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 1 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000013 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 1 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000007 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 1 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000007 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 4 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000246 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 4 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000130 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 4 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000110 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 8 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000134 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 8 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000151 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 8 vect --size 800
ROOT received the reduced dot product: 170346800
Execution time: 0.000187 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$

```

VECTOR SIZE: 500,000,000

```

george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 1 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 5.305913 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 1 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 3.753847 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 1 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 3.670115 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 4 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 3.885150 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 4 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 3.023296 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 4 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 3.082895 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 8 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 3.882954 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 8 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 3.509453 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$ mpirun -n 8 vect --size 500000000
ROOT received the reduced dot product: 9578198922199153536
Execution time: 3.496128 seconds
george@DESKTOP-E24BUDU:~/DistSys24/MPI/w3$

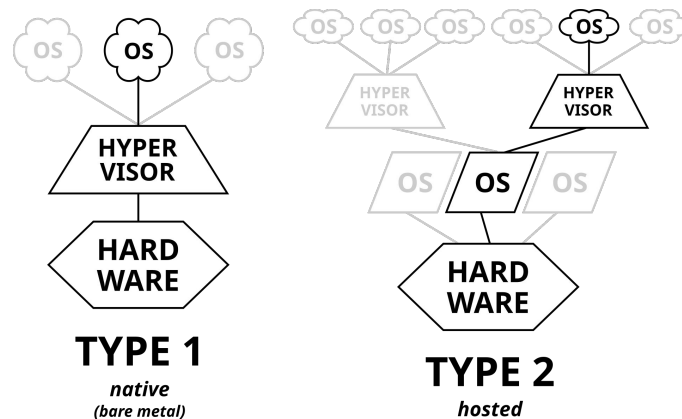
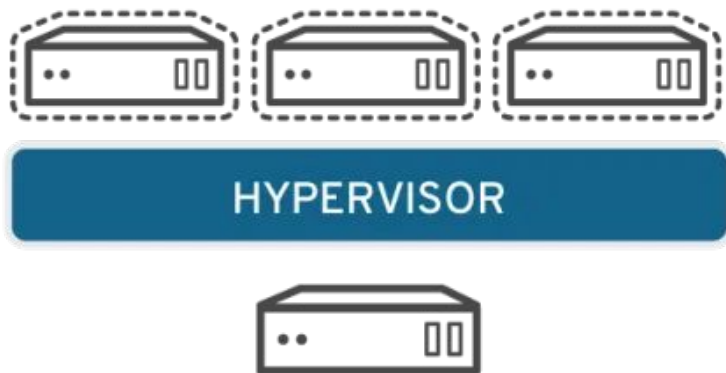
```


Distributed Dot Product (Observations)

- There is a **TRADE-OFF!**
- The more processes we have the more communication is required
- If **COMPUTATION GAIN** < **COMMUNICATION COST**
 - The program will run **SLOWER** with more processes!

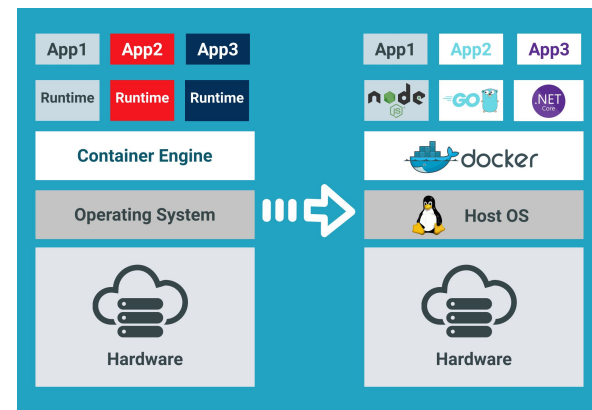
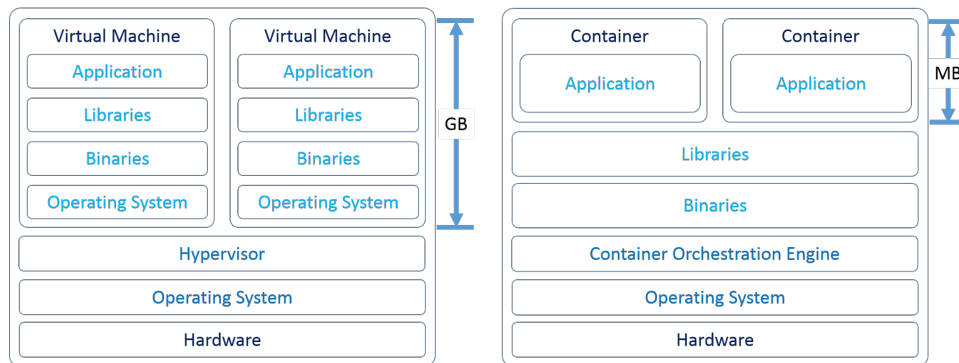
Virtualization: Enabling Cloud Computing

- **Virtualization:**
 - a. creates virtual representations of hardware
 - b. powers cloud computing services by helping organizations manage infrastructure
- **Why?**
 - a. More flexible & **Isolation** secure resource utilisation.
 - b. Painless environment setup & recovery effort (keep everything tidy).
- **Virtual Machine Monitor (VMM, or Hypervisor)**: a type of computer software, or hardware that creates and runs **virtual machines (VMs)**.



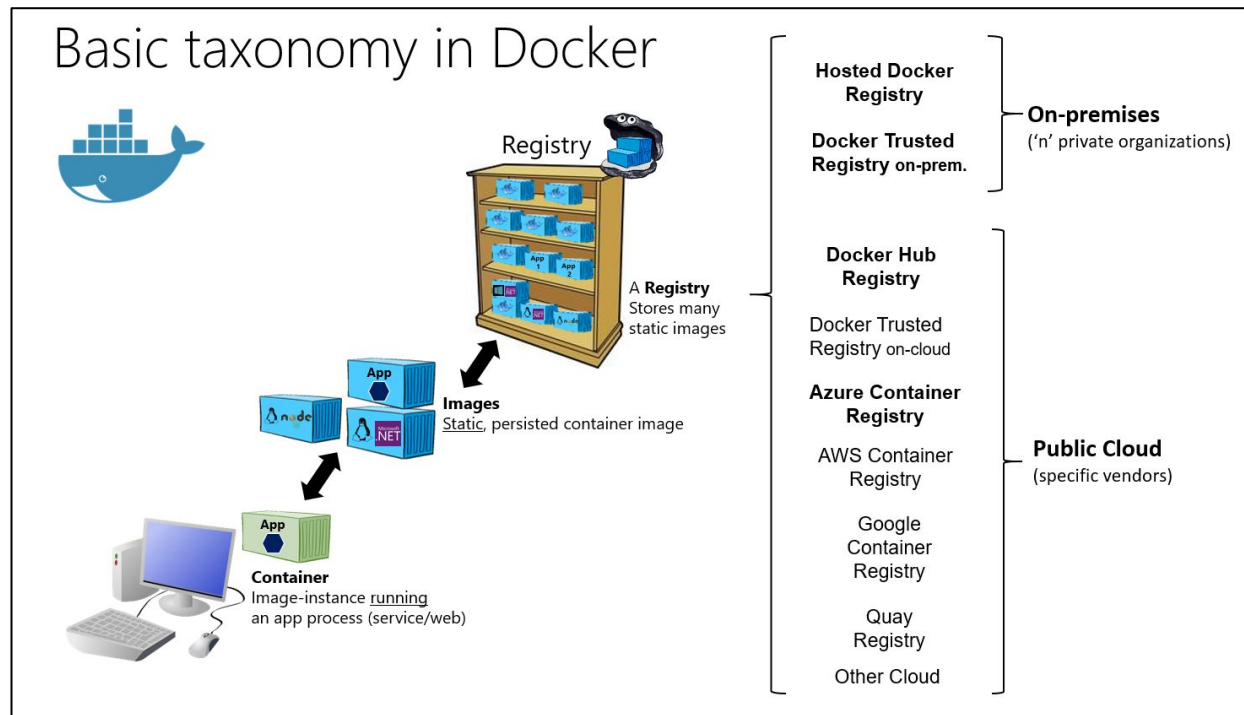
Containerization

- **Container Images:** lightweight packages of application code together with dependencies required to run your software services.
- **Containers (running instance of images) - compared to VMs:**
 - a. **Efficient:**
 - containerized apps share the hosting OS kernel, thus using fewer resources.
 - even easier environment setup.
 - b. **Lightweight:** do not carry OS; only contain necessary libraries & tools.
 - c. **Portability:** OS-independent, thus movable to PCs/VMs/Cloud.



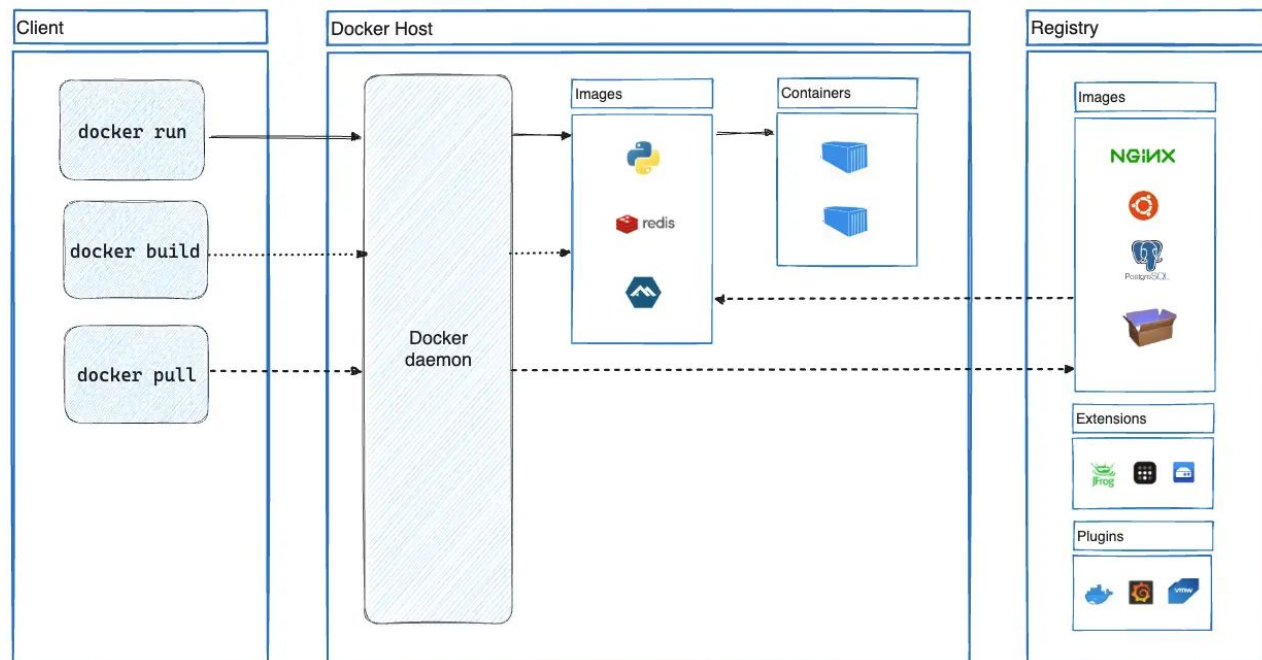
Docker: Basic Concepts

- **Registry**
 - Docker Hub
- **Image**
 - OS like ubuntu
 - apps like postgres
- **Container**
 - image instance



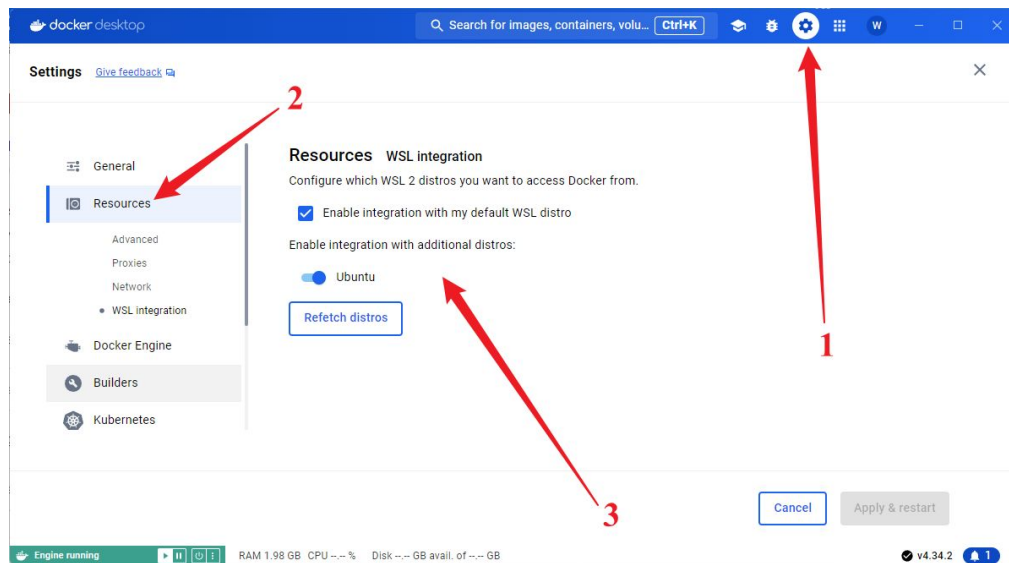
Docker: Architecture

- **Registry**
 - pull images
 - `docker pull`
- **Image**
 - build images
 - requires [Dockerfile](#)
 - `docker build`
- **Container**
 - run images
 - `docker run`
- **Docker CLI/Client**
 - `docker` commands
- **Docker daemon**
 - middleman between Docker Client & Docker objects
- **Docker Desktop**
 - includes the above



Docker: Setup

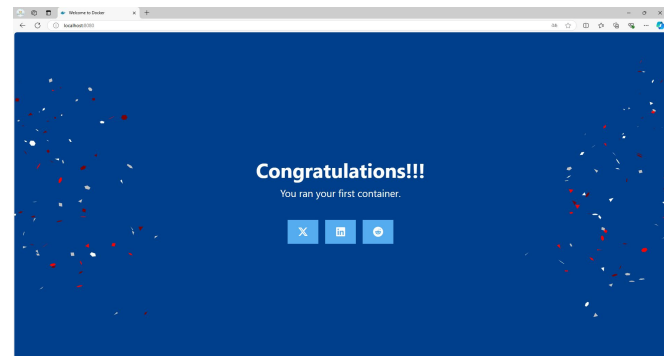
1. Install Docker Desktop: <https://docs.docker.com/get-started/introduction/get-docker-desktop/>.
2. For WSL2 users: enable WSL integration in Docker Desktop settings.
3. [Verify installation](#) by: `docker run -d -p 8080:80 docker/welcome-to-docker`
4. Verify installation by accessing: <http://localhost:8080/>.



```
(base) root@RAI9BOW: # docker run -d -p 8080:80 docker/welcome-to-docker
Unable to find image 'docker/welcome-to-docker:latest' locally
latest: Pulling from docker/welcome-to-docker
96526aa774ef: Pull complete
740091335c74: Pull complete
da9c2e764c5b: Pull complete
ade17ad21ef4: Pull complete
4a6f462c8a69: Pull complete
1324d9977cd2: Pull complete
1b9b96da2c74: Pull complete
5d329b1e101a: Pull complete
Digest: sha256:eadaff45e3c78538087bdd9dc7afafac7e110061bbdd836af4104b10f10ab693
Status: Downloaded newer image for docker/welcome-to-docker:latest
884e1721e36cce24aa19967ed190c49ce76ae71415288638627ecbfefffbce325
```

pull image from registry

run image



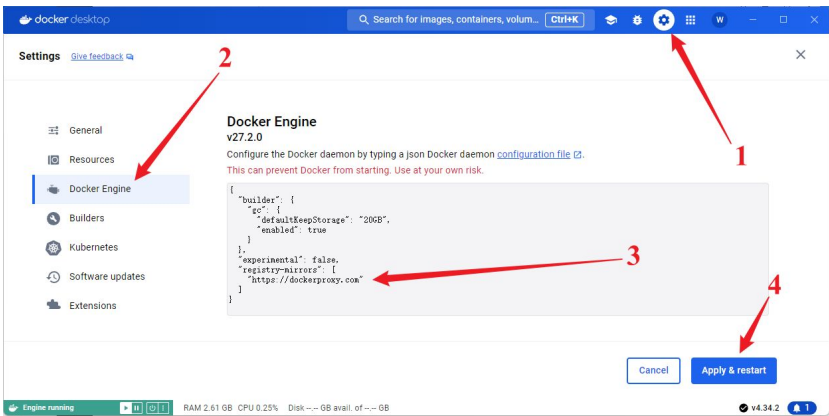
Docker: Setup

(optional) If image pulling encounters timeouts, try to set [registry mirrors](https://dockerproxy.com) in the settings: add <https://dockerproxy.com> as a registry mirror:

```
"registry-mirrors": [
  "https://dockerproxy.com"
]
```

After verification, stop the container and clean the resources.

- 1. Find the corresponding container ID/name: `docker ps`
- 2. Stop the container: `docker container stop [ID/NAME]`
- 3. Remove the container: `docker container rm [ID/NAME]`



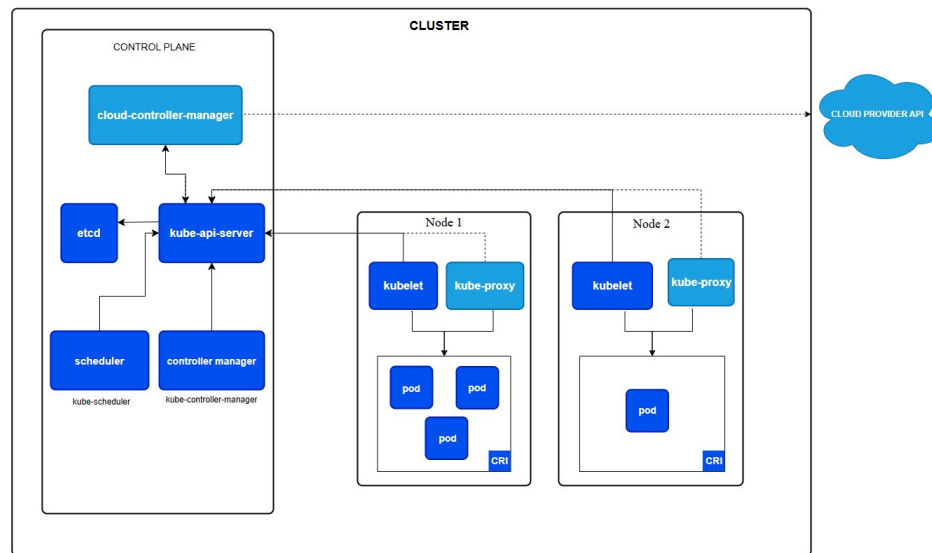
```
(base) root@RAINBOW: /home/rainbow/.nixpkgs/docker_# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
84566a2f3e78	docker/welcome-to-docker	"/docker-entrypoint..."	9 minutes ago	Up 9 minutes	0.0.0.0:8080->80/tcp	sharp_gould

```
(base) root@RAINBOW: /home/rainbow/.nixpkgs/docker_# docker container stop 84566a2f3e78
84566a2f3e78
(base) root@RAINBOW: /home/rainbow/.nixpkgs/docker_# docker container rm 84566a2f3e78
84566a2f3e78
```


Docker Compose, Swarm & Kubernetes

- **Docker Compose**: a tool for defining and running multi-container applications on a **single** host.
 - Requires [compose.yaml](#).
- **Docker - Swarm mode**: an advanced feature for managing a cluster of Docker daemons - running multi-container applications on **multiple** hosts.
- **Kubernetes (K8s)**: a portable, extensible, open source platform for managing containerized workloads and services, that facilitates both **declarative configuration and automation** (more details in the future).



TASK: Test MPI with Docker Compose

Download the sample code zip file from Blackboard and try to run yourself. This sample executes the MPI Hello World program we wrote previously on 2 containers via Docker Compose. It contains:

- a built executable MPI program: `intro`
- `Dockerfile` for `docker build`
- `compose.yaml` for `docker compose`

Follow the following steps:

1. Build an image named `mpi1`: `docker build -t mpi1 .`

```
(base) root@RAINBOW:/home/rainbow/asialab/docker_dist_mpi# ls -lhr
.:
total 12K
-rw-rw-rw- 1 root root 1.2K Sep 24 14:14 Dockerfile
-rw-rw-rw- 1 root root 977 Sep 24 14:10 compose.yaml
drwxrwxrwx 2 root root 4.0K Sep 24 14:10 mpi_files

./mpi_files:
total 16K
-rwxrwxrwx 1 root root 16K Sep 11 10:53 intro
```

```
(base) root@RAINBOW:/home/rainbow/asialab/docker_dist_mpi# docker build -t mpi1 .
[+] Building 1.2s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> == transferring dockerfile: 1.23kB
=> [internal] load metadata for docker.io/library/ubuntu:latest
=> [internal] load .dockerignore
=> == transferring context: 3B
=> [1/6] FROM docker.io/library/ubuntu:latest@sha256:dfe1d870e4dd9fa9c81cbff11188cbidf443913291a9795c7276a899fe294a
=> CACHED [2/6] RUN apt-get update && apt-get install -y build-essential openssh-server openssh-client openssh-keygen
=> CACHED [3/6] RUN echo 'root:root' | chpasswd
=> CACHED [4/6] RUN mkdir -m 0700 /home/mpi -s /bin/bash -q root -S node -u 1001 mpi && echo 'mpi:mpi' | chpasswd
=> CACHED [5/6] RUN mkdir -p /home/mpi/.ssh && ssh-keygen -q -t rsa -N '' -f /home/mpi/.ssh/id_rsa && cat /home/mpi/.ssh/id_
=> CACHED [6/6] WORKDIR /home/mpi
=> exporting to image
=> == exporting layers
=> == writing image sha256-30ec18cc16b13da7a9c898f44c7234d8cc1e1f67b13ae7d8911379a2794c57b3
=> == naming to docker.io/library/mpi1
```

TASK: Test MPI with Docker Compose

Download the sample code zip file from Blackboard and try to run yourself. This sample executes the MPI Hello World program we wrote previously on 2 containers via Docker Compose. Follow the following steps:

1. Build an image named `mpi1`: `docker build -t mpi1 .` ←The dot is important!
2. Start 2 containers: `docker compose up -d`
3. Access the first container: `docker exec -it node1 /bin/bash`
4. Switch to the 'mpi' user: `su mpi`
5. Write host file for `mpirun`:
 - a. `echo node1 slots=2 > host`
 - b. `echo node2 slots=2 >> host`
6. Grant execution permission to the MPI Hello World program (password=mpi): `sudo chmod 777 ./mpi_files/intro`
7. Execute the MPI Hello World program: `mpirun -n 4 --hostfile host ./mpi_files/intro`

```
mpi@node1:~$ mpirun -n 4 --hostfile host ./mpi_files/intro
Hello world from processor node1, rank 0 out of 4 processors
Hello world from processor node1, rank 1 out of 4 processors
Hello world from processor node2, rank 3 out of 4 processors
Hello world from processor node2, rank 2 out of 4 processors
```

```
(base) root@RAINBOW: /home/rainbow/aiialab/docker_dist_mpi# docker compose up -d
[+] Running 3/3
✔ Network docker_dist_mpi_mpi-network Created 0.0s
✔ Container node2 Started 0.1s
✔ Container node1 Started 0.0s
```

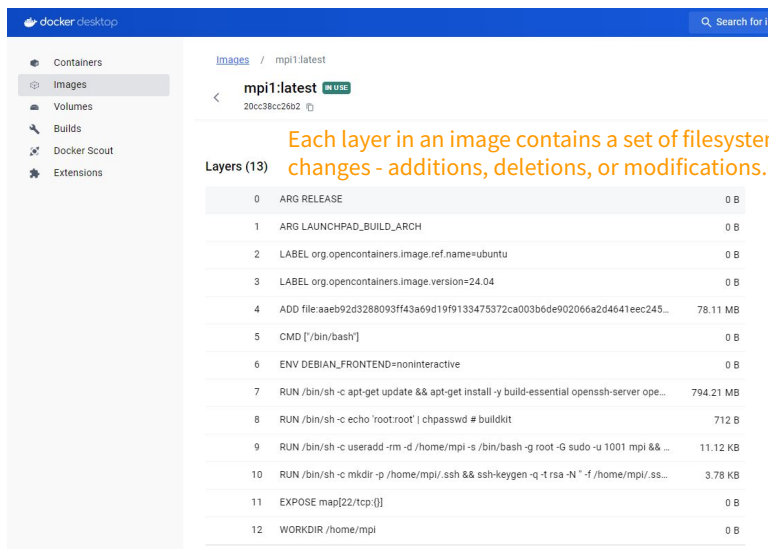
```
(base) root@RAINBOW: /home/rainbow/aiialab/docker_dist_mpi# docker exec -it node1 /bin/bash
root@node1:/home/mpi# su mpi
mpi@node1:~$ echo node1 slots=2 > host
mpi@node1:~$ echo node2 slots=2 >> host
mpi@node1:~$ cat host
node1 slots=2
node2 slots=2
```

```
mpi@node1:~$ sudo chmod 777 ./mpi_files/intro
[sudo] password for mpi:
```

TASK: Test MPI with Docker Compose

Dockerfile:

- **FROM:** define a base image.
- **ENV:** set environment variables.
- **RUN:** execute build commands.
- **EXPOSE:** describe which ports your application is listening on.
- **WORKDIR:** change working directory.



The screenshot shows the Docker Desktop interface. On the left, the 'Images' tab is selected. The main area displays the 'mpi1:latest' image, which is marked as 'NEW'. Below the image name, a table lists the layers of the image, showing the command used to build each layer and its size.

Layer	Command	Size
0	ARG RELEASE	0 B
1	ARG LAUNCHPAD_BUILD_ARCH	0 B
2	LABEL org.opencontainers.image.ref.name=ubuntu	0 B
3	LABEL org.opencontainers.image.version=24.04	0 B
4	ADD file:aaeb92d3288093ff43a69d19f9133475372ca003b6de902066a2d4641eec245...	78.11 MB
5	CMD ["/bin/bash"]	0 B
6	ENV DEBIAN_FRONTEND=noninteractive	0 B
7	RUN /bin/sh -c apt-get update && apt-get install -y build-essential openssh-server ope...	794.21 MB
8	RUN /bin/sh -c echo 'root:root' chpasswd # buildkit	712 B
9	RUN /bin/sh -c useradd -rm -d /home/mpl -s /bin/bash -g root -G sudo -u 1001 mpl && ...	11.12 KB
10	RUN /bin/sh -c mkdir -p /home/mpl/.ssh && ssh-keygen -q -t rsa -N '' -f /home/mpl/.ss...	3.78 KB
11	EXPOSE map[22/tcp:{}]	0 B
12	WORKDIR /home/mpl	0 B

Each layer in an image contains a set of filesystem changes - additions, deletions, or modifications.

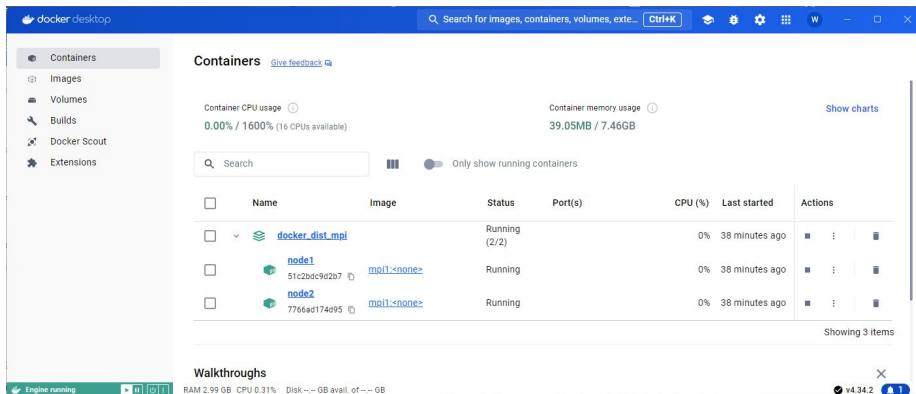
```

1 # Use the official Ubuntu base image
2 FROM ubuntu
3
4 # Set environment variables to prevent interactive prompts during package installations
5 ENV DEBIAN_FRONTEND=noninteractive
6
7 # Update the package list and install necessary packages
8 RUN apt-get update && \
9     apt-get install -y \
10         build-essential \
11         openssh-server \
12         openmpi-bin \
13         openmpi-common \
14         libopenmpi-dev \
15         nano \
16         sudo && \
17         apt-get clean && \
18         rm -rf /var/lib/apt/lists/*
19
20 # set roots passw to root
21 RUN echo 'root:root' | chpasswd
22
23 # Create the mpi user
24 RUN useradd -rm -d /home/mpl -s /bin/bash -g root -G sudo -u 1001 mpl && \
25     echo 'mpl:mpl' | chpasswd
26
27 # Set up SSH configuration for the mpi user
28 RUN mkdir -p /home/mpl/.ssh && \
29     ssh-keygen -q -t rsa -N '' -f /home/mpl/.ssh/id_rsa && \
30     cat /home/mpl/.ssh/id_rsa.pub >> /home/mpl/.ssh/authorized_keys && \
31     # Set ownership to mpl user and correct permissions
32     chown -R mpl:root /home/mpl/.ssh && \
33     chmod 700 /home/mpl/.ssh && \
34     chmod 600 /home/mpl/.ssh/authorized_keys && \
35     echo "StrictHostKeyChecking no" >> /home/mpl/.ssh/config
36
37 # Expose the SSH port
38 EXPOSE 22
39
40 # Set the working directory to ...
41 WORKDIR /home/mpl
  
```

TASK: Test MPI with Docker Compose

compose.yaml:

- **container_name**: used for accessing container by name.
- **hostname**: used for mpirun to recognize hosts by host file.
- **image**: specify the mpi1 image we built.
- **networks**: configure a basic bridge network between 2 hosts.
- **command**: override the default command by the container image.
- **Volumes**: define mount host paths or named volumes that are accessible by service containers.



```
compose.yaml
1  services:
2    node1:
3      container_name: node1
4      hostname: node1
5      image: mpi1
6      networks:
7        - mpi-network
8      # start the ssh service so that other containers can connect to us && wait forever
9      command: /bin/bash -c "
10         service ssh start &&
11         tail -f /dev/null"
12      # allow the container to access the mpi_files directory on the host system from the container directory "/home/mpi/mpi_files/"
13      volumes:
14        - ./mpi_files:/home/mpi/mpi_files/
15
16    node2:
17      container_name: node2
18      hostname: node2
19      image: mpi1
20      networks:
21        - mpi-network
22      # start the ssh service so that other containers can connect to us && wait forever
23      command: /bin/bash -c "
24         service ssh start &&
25         tail -f /dev/null"
26      # allow the container to access the mpi_files directory on the host system from the container directory "/home/mpi/mpi_files/"
27      volumes:
28        - ./mpi_files:/home/mpi/mpi_files/
29
30  networks:
31    mpi-network:
32      driver: bridge
```

TASK: Test MPI with Docker Compose

Download the sample code zip file from Blackboard and try to run yourself. This sample executes the MPI Hello World program we wrote previously on 2 containers via Docker Compose.

As a final step, clean up the resources by: `docker compose down`

That's the end for this task! If you want to play with Docker Compose a bit more, refer to this link: <https://docs.docker.com/compose/gettingstarted/>.

```
(base) root@RAINBOW: /home/rainbow/.vscode/docker_dist_mpi# docker compose down
[+] Running 1/1
✔ Container node1          Removed          10.0s
✔ Container node2          Removed          10.0s
✔ Network docker_dist_mpi_mpi-network Removed          0.2s
```